

Τεχνική Αναφορά — Scikit-learn Agentic RAG Agent

Περιεχόμενα

1. Εισαγωγή
 2. Στόχος του συστήματος
 3. Corpus και θεματικό πεδίο
 4. Αρχιτεκτονική συστήματος
 5. Ingestion και chunking strategies
 6. Metadata schema και filtering
 7. RAG pipeline
 8. Agent architecture
 9. Evaluation methodology
 10. Αποτελέσματα αξιολόγησης
 11. HAIC benchmarking
 12. Παρατηρησιμότητα με Langfuse
 13. Production setup / Docker
 14. Limitations και επόμενα βήματα
 15. Δήλωση συνεισφοράς μελών
-

1. Εισαγωγή

Η παρούσα εργασία αφορά την ανάπτυξη ενός Agentic Retrieval-Augmented Generation (RAG) συστήματος για την υποστήριξη χρηστών που αναζητούν τεκμηριωμένες απαντήσεις στο επίσημο documentation του scikit-learn. Το scikit-learn αποτελεί μία από τις πιο διαδεδομένες βιβλιοθήκες μηχανικής μάθησης στην Python και περιλαμβάνει εκτενή τεκμηρίωση για preprocessing, pipelines, μοντέλα ταξινόμησης, αξιολόγηση μοντέλων, cross-validation και hyperparameter tuning.

Το σύστημα που υλοποιήθηκε δεν βασίζεται αποκλειστικά στις γενικές γνώσεις ενός Large Language Model (LLM), αλλά χρησιμοποιεί RAG pipeline. Συγκεκριμένα, η ερώτηση του χρήστη μετατρέπεται σε embedding, αναζητούνται σχετικά αποσπάσματα από το corpus μέσω Qdrant vector store και η τελική απάντηση παράγεται με βάση το ανακτημένο context. Με αυτόν τον τρόπο, οι απαντήσεις είναι περισσότερο grounded στο διαθέσιμο υλικό και συνοδεύονται από πηγές.

Επιπλέον, υλοποιήθηκε agentic επίπεδο πάνω από το RAG pipeline. Αρχικά δημιουργήθηκε ένας rule-based routing agent ως baseline, ενώ στη συνέχεια προστέθηκε LangGraph ReAct agent, ο οποίος επιλέγει εργαλείο ανάκτησης με πιο agentic λογική. Το σύστημα αξιολογήθηκε με golden test set 30 ερωτήσεων, μετρικές retrieval και LLM-as-judge, Tool Call Accuracy, HAIC benchmarking και Langfuse tracing για παρατηρησιμότητα.

2. Στόχος του συστήματος

Στόχος του συστήματος είναι να βοηθά αρχάριους και μεσαίου επιπέδου χρήστες της Python και της μηχανικής μάθησης να βρίσκουν γρήγορα και τεκμηριωμένα απαντήσεις σε ερωτήματα που σχετίζονται με supervised classification workflows στο scikit-learn.

Το σύστημα υποστηρίζει ερωτήσεις όπως:

- τι είναι το feature scaling και πότε χρησιμοποιείται,
- πώς δημιουργείται ένα Pipeline με StandardScaler και LogisticRegression,
- γιατί το cross-validation είναι χρήσιμο στην αξιολόγηση μοντέλων,
- πώς χρησιμοποιείται το GridSearchCV για hyperparameter tuning,
- ποια metrics είναι κατάλληλα για imbalanced classification,
- ποιες παράμετροι του RandomForestClassifier επηρεάζουν την πολυπλοκότητα του μοντέλου.

Ο βασικός στόχος δεν ήταν απλώς η παραγωγή απαντήσεων, αλλά η δημιουργία ενός ολοκληρωμένου συστήματος που περιλαμβάνει corpus ingestion, chunking, vector indexing, retrieval, grounded generation, agent routing, evaluation, HAIC benchmarking, observability και αναπαραγώγιμη εκτέλεση μέσω README και Docker/Qdrant setup.

3. Corpus και θεματικό πεδίο

Το corpus της εργασίας αποτελείται από 10 επιλεγμένες σελίδες του επίσημου scikit-learn documentation. Το θεματικό πεδίο περιορίστηκε σε supervised classification workflows, ώστε το corpus να είναι εστιασμένο αλλά αρκετά πλούσιο για να υποστηρίζει διαφορετικούς τύπους ερωτημάτων.

Οι πηγές επιλέχθηκαν ώστε να καλύπτουν ένα πλήρες workflow μηχανικής μάθησης: εισαγωγή στο scikit-learn, preprocessing, pipelines, train/test split, cross-validation, hyperparameter tuning, evaluation metrics, Logistic Regression και Random Forest.

3.1 Corpus sources

ID	Θέμα	Περιγραφή
D0 1	Γενική εισαγωγή στο scikit-learn	Περιλαμβάνει βασικές έννοιες όπως fitting, predicting, transformers, pipelines, evaluation και parameter search.
D0 2	Preprocessing / scaling	Καλύπτει preprocessing τεχνικές, όπως StandardScaler, και εξηγεί γιατί το scaling επηρεάζει αρκετούς αλγορίθμους.
D0 3	Pipelines	Περιγράφει pipelines και composite estimators, με έμφαση στη σωστή οργάνωση ML workflows και στη μείωση data leakage.
D0 4	Train/test split	Καλύπτει τη χρήση του train_test_split για διαχωρισμό δεδομένων σε train και test sets.
D0 5	Cross-validation	Περιγράφει την αξιολόγηση μοντέλων με cross-validation και folds.
D0 6	Hyperparameter tuning	Καλύπτει GridSearchCV, RandomizedSearchCV και επιλογή παραμέτρων.

ID	Θέμα	Περιγραφή
D07	Metrics	Περιλαμβάνει metrics όπως accuracy, precision, recall, F1-score και scoring.
D08	Logistic Regression	Περιγράφει τον LogisticRegression classifier και βασικούς παραμέτρους regularization.
D09	Random Forests	Περιγράφει ensemble methods και random forests σε εννοιολογικό επίπεδο.
D10	RandomForestClassifier	Περιλαμβάνει την API σελίδα του RandomForestClassifier και παραμέτρους όπως max_depth, min_samples_split, max_features και ccp_alpha.

Οι παραπάνω πηγές οργανώθηκαν στο αρχείο `data/corpus_sources.csv`, ώστε κάθε document και κάθε μελλοντικό chunk να μπορεί να συνδεθεί με την αρχική του πηγή μέσω metadata.

3.2 Τύποι ερωτημάτων που υποστηρίζει το corpus

Το corpus υποστηρίζει τουλάχιστον τρεις βασικούς τύπους ερωτημάτων:

1. **Conceptual questions** Ερωτήσεις κατανόησης εννοιών, όπως τι είναι το feature scaling, τι σημαίνει cross-validation ή ποια είναι η διαφορά μεταξύ precision, recall και F1-score.
2. **Procedural / how-to questions** Ερωτήσεις που ζητούν πρακτικά βήματα, όπως πώς δημιουργείται ένα Pipeline με StandardScaler και LogisticRegression ή πώς χρησιμοποιείται το GridSearchCV.
3. **Decision-support questions** Ερωτήσεις που βοηθούν τον χρήστη να επιλέξει κατάλληλη μέθοδο ή metric, όπως πότε είναι προτιμότερο το F1-score αντί για accuracy ή πότε μπορεί να χρησιμοποιηθεί RandomForestClassifier αντί για LogisticRegression.

Το corpus θεωρήθηκε κατάλληλο για RAG επειδή έχει τεχνικό βάθος, καθαρή δομή, επίσημη τεκμηρίωση και επαρκή ποικιλία ώστε να επιτρέπει σύγκριση διαφορετικών ingestion και retrieval strategies.

4. Αρχιτεκτονική συστήματος

Το σύστημα υλοποιήθηκε ως end-to-end Agentic RAG pipeline. Η συνολική ροή ξεκινά από τη συλλογή του corpus, συνεχίζει με δύο διαφορετικές στρατηγικές chunking, αποθηκεύει embeddings σε Qdrant vector store και στη συνέχεια εκτελεί retrieval και grounded generation. Πάνω από το βασικό RAG pipeline υλοποιήθηκε agent layer, το οποίο επιλέγει κατάλληλο retrieval εργαλείο ανάλογα με την ερώτηση του χρήστη.

Η αρχιτεκτονική του συστήματος μπορεί να συνοψιστεί ως εξής:

```

Scikit-learn documentation corpus
|
v
Corpus ingestion
|
v
Chunking strategies
(v1_fixed / v2_structured)
|
v
Embeddings
(text-embedding-3-small)
|
v
Qdrant vector store
(sklearn_rag_v1_fixed / sklearn_rag_v2_structured)
|
v
Retriever
|
v
Context construction
|
v
Grounded generation
(gpt-4.1-mini)
|
v
Agent layer
(rule-based agent / LangGraph ReAct agent)
|
v
Evaluation, HAIC benchmarking, Langfuse tracing

```

Τα βασικά components του συστήματος είναι:

- **Corpus ingestion:** συλλογή και αποθήκευση των επιλεγμένων scikit-learn documentation pages.
- **Chunking:** υλοποίηση δύο στρατηγικών, fixed-size chunking και structured/section-based chunking.
- **Embeddings:** μετατροπή των chunks και των ερωτήσεων σε vector representations μέσω `text-embedding-3-small`.
- **Vector store:** αποθήκευση και αναζήτηση embeddings στο Qdrant.
- **RAG pipeline:** ανάκτηση σχετικών chunks, δημιουργία context και παραγωγή απάντησης με grounding.
- **Agent layer:** επιλογή εργαλείου ανάκτησης είτε με rule-based routing είτε με LangGraph ReAct agent.
- **Evaluation:** αξιολόγηση retrieval, generation, tool selection και HAIC metrics.
- **Observability:** καταγραφή demo trace μέσω Langfuse.

Η υλοποίηση οργανώθηκε σε διακριτούς φακέλους: `src/ingestion/`, `src/vectorstore/`, `src/rag/`, `src/agent/`, `src/evaluation/`, ενώ τα δεδομένα και τα αποτελέσματα αποθηκεύτηκαν στους φακέλους `data/`, `evaluation/` και `outputs/`.

5. Ingestion και chunking strategies

Για το corpus ingestion χρησιμοποιήθηκαν 10 επιλεγμένες σελίδες του scikit-learn documentation. Οι πηγές καταγράφηκαν στο `data/corpus_sources.csv` και αποθηκεύτηκαν τοπικά στον φάκελο `data/raw/` μαζί με τα αντίστοιχα metadata αρχεία.

Η διαδικασία ingestion υλοποιήθηκε στο script:

```
src/ingestion/download_corpus.py
```

Το script κατεβάζει τις σελίδες, καθαρίζει το HTML περιεχόμενο και αποθηκεύει τα κείμενα ως `.txt` αρχεία, καθώς και τα metadata ως `.json` αρχεία. Έτσι, το corpus παραμένει διαθέσιμο τοπικά και μπορεί να χρησιμοποιηθεί ξανά χωρίς απαραίτητα να γίνει νέο download.

Για τη δημιουργία chunks υλοποιήθηκαν δύο στρατηγικές:

5.1 v1_fixed — fixed-size chunking

Η πρώτη στρατηγική ήταν το `v1_fixed`, δηλαδή fixed-size chunking. Σε αυτή την προσέγγιση, το κείμενο χωρίζεται σε chunks σταθερού μεγέθους, με overlap μεταξύ διαδοχικών chunks. Η λογική αυτής της στρατηγικής είναι απλή και λειτουργεί ως baseline για τη σύγκριση με πιο δομημένες τεχνικές.

Το αντίστοιχο script είναι:

```
src/ingestion/chunk_fixed.py
```

Η παραμετροποίηση βρίσκεται στο:

```
configs/v1_fixed_chunking.json
```

Το output αποθηκεύτηκε στο:

```
data/processed/v1_fixed_chunks.jsonl
```

Η στρατηγική αυτή παράγαγε **402 chunks**.

5.2 v2_structured — structured / section-based chunking

Η δεύτερη στρατηγική ήταν το `v2_structured`, δηλαδή structured ή section-based chunking. Σε αυτή την προσέγγιση, το κείμενο χωρίζεται με βάση τη δομή των documentation pages, όπως headings και sections. Ο στόχος ήταν τα chunks να διατηρούν καλύτερα το νοηματικό τους πλαίσιο και να αντιστοιχούν σε πιο συνεκτικές ενότητες πληροφορίας.

Το αντίστοιχο script είναι:

```
src/ingestion/chunk_structured.py
```

Η παραμετροποίηση βρίσκεται στο:

```
configs/v2_structured_chunking.json
```

Το output αποθηκεύτηκε στο:

```
data/processed/v2_structured_chunks.jsonl
```

Η στρατηγική αυτή παρήγαγε **375 chunks**.

5.3 Σύγκριση chunking strategies

Strategy	Περιγραφή	Output file	Αριθμός chunks
v1_fixed	Fixed-size chunking με overlap	data/processed/ v1_fixed_chunks.jsonl	402
v2_structured	Section-based chunking με βάση headings/sections	data/processed/ v2_structured_chunks.jsonl	375

Η σύγκριση των δύο στρατηγικών έδειξε ότι το structured chunking ήταν πιο αποτελεσματικό για το συγκεκριμένο corpus. Παρότι παρήγαγε λιγότερα chunks, τα chunks ήταν πιο νοηματικά συνεκτικά και οδήγησαν σε καλύτερα retrieval αποτελέσματα, ειδικά στο top-1 retrieval.

6. Metadata schema και filtering

Κάθε chunk συνοδεύτηκε από metadata, ώστε το σύστημα να μπορεί να συνδέει κάθε ανακτημένο απόσπασμα με την αρχική πηγή του και να υποστηρίζει metadata filtering.

Το metadata schema περιλαμβάνει πεδία όπως:

Πεδίο	Περιγραφή
chunk_id	Μοναδικό αναγνωριστικό του chunk
doc_id	Αναγνωριστικό αρχικού document, π.χ. D01-D10
title	Τίτλος της πηγής
topic	Θεματική κατηγορία του document/chunk
url	URL της αρχικής σελίδας του scikit-learn documentation
section	Ενότητα ή section από την οποία προέρχεται το chunk
heading_level	Επίπεδο heading, όπου υπάρχει
chunking_strategy	Στρατηγική chunking, π.χ. v1_fixed ή v2_structured
text	Το περιεχόμενο του chunk

Η χρήση metadata ήταν σημαντική για δύο λόγους. Πρώτον, επέτρεψε την εμφάνιση πηγών στις τελικές απαντήσεις. Δεύτερον, επέτρεψε την υλοποίηση metadata-filtered retrieval για ερωτήσεις που αφορούν συγκεκριμένο topic.

Για παράδειγμα, στην ερώτηση:

```
Which RandomForestClassifier parameters can control model complexity?
```

το σύστημα μπορεί να χρησιμοποιήσει topic filter:

```
random_forest_classifier
```

ώστε η αναζήτηση να περιοριστεί στα chunks που προέρχονται από την πηγή D10. Αυτό βελτίωσε την ακρίβεια της ανάκτησης σε API-specific ερωτήσεις, όπου η απλή semantic search μπορεί να επιστρέψει σχετικά αλλά λιγότερο κατάλληλα chunks από γενικότερες σελίδες, όπως η ενότητα Random Forests.

Το metadata filtering υλοποιήθηκε μέσω Qdrant payload filtering και χρησιμοποιείται από το εργαλείο:

```
metadata_filtered_retriever
```

Το απλό retrieval εργαλείο:

```
rag_retriever
```

αναζητά στο συνολικό corpus χωρίς topic filter.

7. RAG pipeline

Το βασικό RAG pipeline υλοποιήθηκε στο αρχείο:

```
src/rag/rag_answer.py
```

Η ροή του pipeline είναι η εξής:

1. Η ερώτηση του χρήστη μετατρέπεται σε embedding.
2. Το embedding χρησιμοποιείται για semantic search στο Qdrant.
3. Ανακτώνται τα top-k σχετικά chunks.
4. Τα chunks μορφοποιούνται σε context.
5. Το context και η ερώτηση δίνονται στο LLM.
6. Το LLM παράγει grounded απάντηση βασισμένη μόνο στο διαθέσιμο context.
7. Η τελική απάντηση επιστρέφεται μαζί με τις πηγές.

Η βασική λογική μπορεί να συνοψιστεί ως:

```
question
  → embedding
  → vector search in Qdrant
  → retrieved chunks
  → context construction
  → grounded LLM answer
  → answer + sources
```

Για embeddings χρησιμοποιήθηκε το μοντέλο:

```
text-embedding-3-small
```

Για generation χρησιμοποιήθηκε:

```
gpt-4.1-mini
```

Το σύστημα prompt δίνει ρητές οδηγίες στο LLM να απαντά μόνο με βάση το ανακτημένο context. Αν η πληροφορία δεν υπάρχει στο context, το σύστημα πρέπει να το δηλώνει αντί να επινοεί απάντηση. Αυτό ελέγχθηκε και με out-of-context ερώτηση, όπως:

```
Who won the FIFA World Cup in 2022?
```

η οποία δεν ανήκει στο corpus του scikit-learn. Η συμπεριφορά αυτή είναι σημαντική για τη μείωση hallucinations.

Τα βασικά RAG scripts είναι:

Script	Ρόλος
src/rag/rag_answer.py	Εκτέλεση single-question RAG απάντησης
src/rag/test_retrieval.py	Έλεγχος απλής semantic retrieval
src/rag/test_filtered_retrieval.py	Έλεγχος metadata-filtered retrieval
src/rag/run_golden_rag.py	Εκτέλεση RAG πάνω στο golden test set
src/rag/llm_judge_evaluation.py	Αξιολόγηση απαντήσεων με LLM-as-judge

Για την αποθήκευση των embeddings δημιουργήθηκαν δύο Qdrant collections:

Collection	Chunking strategy
sklearn_rag_v1_fixed	Fixed-size chunking
sklearn_rag_v2_structured	Structured/section-based chunking

Η ύπαρξη δύο collections επέτρεψε τη συστηματική σύγκριση των δύο ingestion strategies στο ίδιο σύνολο ερωτήσεων.

8. Agent architecture

Πάνω από το βασικό RAG pipeline υλοποιήθηκε agent layer, με στόχο το σύστημα να μπορεί να επιλέγει διαφορετικό retrieval εργαλείο ανάλογα με το είδος της ερώτησης. Η εργασία περιλαμβάνει δύο agentic εκδοχές:

1. έναν rule-based routing agent ως baseline,
2. έναν LangGraph ReAct agent ως βελτιωμένη agentic υλοποίηση.

8.1 Rule-based routing agent

Ο αρχικός agent υλοποιήθηκε στο αρχείο:

```
src/agent/react_agent.py
```

Η λειτουργία του βασίζεται σε deterministic κανόνες. Συγκεκριμένα, η ερώτηση του χρήστη ελέγχεται για keywords ή φράσεις που υποδεικνύουν ότι χρειάζεται συγκεκριμένο topic filter. Αν η ερώτηση αφορά γενικό workflow ή εννοιολογική πληροφορία, χρησιμοποιείται το απλό retrieval tool. Αν η ερώτηση αφορά συγκεκριμένο API topic, όπως `RandomForestClassifier`, χρησιμοποιείται metadata-filtered retrieval.

Τα δύο βασικά εργαλεία είναι:

Tool	Περιγραφή
<code>rag_retriever</code>	Εκτελεί αναζήτηση στο συνολικό corpus χωρίς metadata filter.
<code>metadata_filtered_retriever</code>	Εκτελεί αναζήτηση με topic filter, ώστε να περιορίσει την ανάκτηση σε συγκεκριμένη θεματική ενότητα.

Για παράδειγμα, στην ερώτηση:

```
Which RandomForestClassifier parameters can control model complexity?
```

ο rule-based agent επιλέγει το εργαλείο:

```
metadata_filtered_retriever
```

με:

```
topic_filter = random_forest_classifier
```

Η επιλογή αυτή οδηγεί στην ανάκτηση chunks από την πηγή D10, δηλαδή την API σελίδα του `RandomForestClassifier`.

Ο rule-based agent λειτούργησε ως σταθερό baseline, επειδή η συμπεριφορά του είναι προβλέψιμη και εύκολα αξιολογήσιμη. Ωστόσο, επειδή η επιλογή εργαλείου βασίζεται σε κανόνες και όχι σε LLM reasoning, δεν αποτελεί πλήρη ReAct agent με την αυστηρή έννοια. Για αυτόν τον λόγο προστέθηκε και δεύτερη agentic εκδοχή με LangGraph.

8.2 LangGraph ReAct agent

Η βελτιωμένη agentic εκδοχή υλοποιήθηκε στο αρχείο:

```
src/agent/langgraph_agent.py
```

και χρησιμοποιεί τα tool wrappers που ορίστηκαν στο:

```
src/agent/tool_definitions.py
```

Ο LangGraph agent χρησιμοποιεί ReAct λογική, όπου το LLM επιλέγει ποιο εργαλείο ανάκτησης πρέπει να χρησιμοποιηθεί με βάση την ερώτηση. Τα εργαλεία που εκτίθενται στον agent είναι τα ίδια με αυτά του baseline:

Tool	Περιγραφή
rag_retriever	Γενική αναζήτηση στο corpus.
metadata_filtered_retriever	Αναζήτηση με metadata/topic filter.

Η προσθήκη του LangGraph agent έγινε με additive τρόπο, χωρίς να αλλάξει το ήδη λειτουργικό RAG pipeline. Ο παλιός rule-based agent διατηρήθηκε ως baseline και fallback, ενώ ο νέος LangGraph agent προστέθηκε ως ξεχωριστό entry point.

Ο LangGraph agent μπορεί να εκτελεστεί με:

```
python src/agent/langgraph_agent.py --question "Which RandomForestClassifier parameters can control model complexity?"
```

Υποστηρίζεται επίσης hybrid mode:

```
python src/agent/langgraph_agent.py --question "Which RandomForestClassifier parameters can control model complexity?" --router hybrid
```

Στο demo με την ερώτηση για τις παραμέτρους του `RandomForestClassifier`, ο LangGraph agent επέλεξε σωστά το εργαλείο `metadata_filtered_retriever`, χρησιμοποίησε το topic filter `random_forest_classifier` και ανέκτησε chunks από την πηγή D10.

8.3 Σύγκριση rule-based και LangGraph agent

Χαρακτηριστικό	Rule-based agent	LangGraph ReAct agent
Routing	Deterministic keyword rules	LLM-based tool selection
Tools	2 retrieval tools	2 retrieval tools
Metadata filtering	Ναι	Ναι
ReAct λογική	Περιορισμένη / conceptual	Ναι, μέσω LangGraph
Σταθερότητα	Υψηλή	Εξαρτάται από LLM routing
Ρόλος στην εργασία	Baseline και fallback	Improved agentic version

Η ύπαρξη και των δύο agents επιτρέπει πιο καθαρή τεκμηρίωση της εξέλιξης του project: πρώτα υλοποιήθηκε σταθερή baseline λογική και στη συνέχεια προστέθηκε πιο agentic LangGraph εκδοχή.

9. Evaluation methodology

Η αξιολόγηση του συστήματος έγινε σε πολλαπλά επίπεδα, ώστε να εξεταστεί τόσο η ποιότητα της ανάκτησης όσο και η ποιότητα των απαντήσεων και η agentic συμπεριφορά.

Το βασικό evaluation set ήταν ένα golden test set 30 ερωτήσεων, αποθηκευμένο στο αρχείο:

```
evaluation/golden_test_set_v0-3.xlsx
```

Οι ερωτήσεις καλύπτουν conceptual, procedural και decision-support περιπτώσεις. Για κάθε ερώτηση υπάρχουν αναμενόμενες πηγές και ενδεικτικές απαντήσεις, ώστε να μπορεί να αξιολογηθεί αν το σύστημα ανακτά το σωστό context και αν παράγει απάντηση συμβατή με το corpus.

Η αξιολόγηση περιλάμβανε:

1. retrieval evaluation,
2. comparison μεταξύ `v1_fixed` και `v2_structured`,
3. LLM-as-judge evaluation,
4. Tool Call Accuracy,
5. HAIC benchmarking.

9.1 Retrieval evaluation

Η retrieval αξιολόγηση εξέτασε αν η αναμενόμενη πηγή εμφανίζεται στα ανακτημένα chunks. Χρησιμοποιήθηκαν δύο βασικές μετρικές:

Metric	Περιγραφή
Expected source in top-k	Αν η αναμενόμενη πηγή βρίσκεται στα top-k retrieved chunks.
Expected source top-1	Αν η αναμενόμενη πηγή είναι το πρώτο retrieved result.

Η σύγκριση έγινε μεταξύ των δύο Qdrant collections:

```
sklearn_rag_v1_fixed  
sklearn_rag_v2_structured
```

9.2 LLM-as-judge evaluation

Για την αξιολόγηση των απαντήσεων χρησιμοποιήθηκε LLM-as-judge προσέγγιση. Το judge αξιολόγησε τις απαντήσεις με βάση πέντε διαστάσεις:

Metric	Περιγραφή
Answer Relevancy	Κατά πόσο η απάντηση απαντά στην ερώτηση.
Faithfulness	Κατά πόσο η απάντηση είναι πιστή στο ανακτημένο context.
Context Precision	Κατά πόσο το retrieved context είναι σχετικό.
Context Recall	Κατά πόσο το context καλύπτει την απαιτούμενη πληροφορία.
Expected Answer Alignment	Κατά πόσο η απάντηση συμφωνεί με την αναμενόμενη απάντηση.

Η αξιολόγηση αποθηκεύτηκε σε αρχεία όπως:

```
evaluation/llm_judge_results_v2_structured_full.csv  
evaluation/llm_judge_summary_v2_structured_full.md
```

9.3 Tool Call Accuracy

Για το agentic κομμάτι αξιολογήθηκε η επιλογή εργαλείου μέσω Tool Call Accuracy. Η μετρική εξετάζει αν ο agent επέλεξε το αναμενόμενο retrieval tool και, όπου χρειάζεται, το σωστό topic filter.

Το σχετικό script είναι:

```
src/agent/evaluate_tool_calls.py
```

Το output αποθηκεύτηκε στα:

```
evaluation/tool_call_accuracy_results.csv  
evaluation/tool_call_accuracy_summary.md
```

Η συγκεκριμένη μετρική εφαρμόστηκε στον rule-based router, ο οποίος λειτουργεί ως σταθερό baseline για agentic routing.

10. Αποτελέσματα αξιολόγησης

10.1 Retrieval results

Η σύγκριση των δύο ingestion strategies έδειξε ότι η structured στρατηγική (v2_structured) είχε καλύτερη retrieval επίδοση από τη fixed-size baseline στρατηγική (v1_fixed).

Metric	v1_fixed	v2_structured
Expected source in top-k	93.33%	96.67%
Expected source top-1	73.33%	80.00%

Το αποτέλεσμα αυτό δείχνει ότι η διατήρηση της δομής των documentation pages βοήθησε την ανάκτηση. Παρότι το v2_structured δημιούργησε λιγότερα chunks από το

`v1_fixed`, τα chunks ήταν πιο συνεκτικά και πιο κοντά στις πραγματικές ενότητες του documentation.

Η βελτίωση στο top-1 retrieval είναι ιδιαίτερα σημαντική, επειδή δείχνει ότι το σωστό document εμφανίζεται συχνότερα ως πρώτο αποτέλεσμα. Αυτό επηρεάζει άμεσα την ποιότητα του context που δίνεται στο LLM.

10.2 LLM-as-judge results

Η LLM-as-judge αξιολόγηση έδειξε υψηλές επιδόσεις για το `v2_structured` pipeline. Οι μέσοι όροι των επιμέρους μετρικών κυμάνθηκαν περίπου στο εύρος 4.73–4.97/5, ανάλογα με τη μετρική.

Evaluation aspect	Συνολική εικόνα
Answer Relevancy	Υψηλή
Faithfulness	Υψηλή
Context Precision	Υψηλή
Context Recall	Υψηλή
Expected Answer Alignment	Υψηλή

Τα αποτελέσματα αυτά δείχνουν ότι το σύστημα παρήγαγε απαντήσεις σχετικές με τις ερωτήσεις, grounded στο ανακτημένο context και συμβατές με τις αναμενόμενες απαντήσεις του golden test set.

10.3 Tool Call Accuracy

Η αξιολόγηση του rule-based agent έδειξε Tool Call Accuracy ίση με 100% στο golden test set των 30 ερωτήσεων. Αυτό σημαίνει ότι, για τις ερωτήσεις του test set, ο deterministic router επέλεξε το αναμενόμενο εργαλείο και, όπου χρειαζόταν, το σωστό topic filter.

Το αποτέλεσμα αυτό πρέπει να ερμηνευθεί προσεκτικά. Επειδή ο rule-based agent και το expected routing βασίζονται σε προκαθορισμένη λογική, η μετρική δείχνει κυρίως ότι η υλοποίηση των κανόνων είναι συνεπής ως baseline. Για αυτόν τον λόγο, ο LangGraph agent παρουσιάζεται ως βελτιωμένη agentic εκδοχή, ενώ ο rule-based router παραμένει χρήσιμος για σταθερότητα και fallback.

10.4 Failure analysis

Ένα χαρακτηριστικό failure case ήταν η ερώτηση Q28:

Which RandomForestClassifier parameters can control model complexity?

Χωρίς metadata filtering, το semantic retrieval μπορούσε να ανακτήσει chunks από γενικότερες πηγές για Random Forests, αντί για την πιο κατάλληλη API σελίδα του `RandomForestClassifier`. Με την προσθήκη topic filter `random_forest_classifier`, η ανάκτηση περιορίστηκε στην D10 πηγή και βελτιώθηκε η ακρίβεια του context.

Αυτό το παράδειγμα δείχνει γιατί το metadata schema και το metadata filtering ήταν σημαντικά για το σύστημα. Η απλή semantic similarity δεν είναι πάντα αρκετή σε τεχνικά corpus, ειδικά όταν υπάρχουν κοντινά αλλά διαφορετικά topics.

11. HAIC benchmarking

Για την αξιολόγηση της Human-AI Collaboration διάστασης εφαρμόστηκε HAIC benchmarking. Η εργασία περιλαμβάνει professor-style HAIC artifact με schema:

```
haic.decisions_artifact.v1
```

Η υλοποίηση βρίσκεται στο script:

```
src/evaluation/haic_professor_benchmark.py
```

Το benchmark εκτελέστηκε πάνω στις 30 ερωτήσεις του golden test set και δημιούργησε 150 logged events. Τα events περιλαμβάνουν βήματα όπως human query, AI routing, tool call, AI response και accept/reject proxy.

Τα βασικά outputs είναι:

```
evaluation/haic_events_v2_structured.jsonl  
evaluation/haic_event_summary_v2_structured.csv  
evaluation/haic_metrics_v2_structured.json  
evaluation/haic_metrics_v2_structured.md
```

11.1 HAIC configuration

Παράμετρος	Τιμή
Sessions	30
Logged events	150
baseline_s	30.00
rt_max_s	30.00

11.2 HAIC metrics

Metric	Value	Interpretation
EL	0.000	Effort loss compared to baseline
Tr	0.967	Fraction of accepted AI responses
HCL	0.883	Higher value indicates lower cognitive load
F	51.404	Interaction events per minute
A	0.197	Adaptability across early vs late decisions
D	1.167 s	Mean duration per decision event
EfficiencyScore	1.000	Composite efficiency score
S	N/A	Excluded because no surrogate simulation was used

Τα αποτελέσματα δείχνουν ότι το σύστημα κατατάσσεται στα diagnostics ως:

EL × Tr: Efficient & Trusted
HCL × F: Smooth collaboration

Η ερμηνεία είναι ότι, με βάση το offline benchmark, το σύστημα απαντά αποτελεσματικά και οι απαντήσεις θεωρούνται αποδεκτές στις περισσότερες περιπτώσεις.

11.3 Περιορισμός HAIC evaluation

Ένας σημαντικός περιορισμός είναι ότι δεν πραγματοποιήθηκε πραγματικό user study. Το human accept/reject event προσεγγίστηκε offline με proxy: μία απάντηση θεωρείται accepted όταν η αναμενόμενη πηγή εντοπίζεται στα retrieved chunks.

Αυτό σημαίνει ότι οι HAIC μετρικές δεν πρέπει να ερμηνευθούν ως πραγματική ανθρώπινη αξιολόγηση, αλλά ως offline approximation της Human-AI Collaboration ποιότητας. Παρ' όλα αυτά, το HAIC benchmark ήταν χρήσιμο για τη συστηματική αποτύπωση των decision events, της αποδοχής απαντήσεων και της συνολικής συνεργατικής συμπεριφοράς του συστήματος.

12. Παρατηρησιμότητα με Langfuse

Για την παρατηρησιμότητα του συστήματος χρησιμοποιήθηκε Langfuse demo script. Η υλοποίηση βρίσκεται στο αρχείο:

```
src/agent/traced_agent_demo.py
```

Το script εκτελεί demo ερώτηση σχετική με το `RandomForestClassifier` και καταγράφει βήματα του agent/RAG flow, όπως embedding, retrieval και generation. Το αποτέλεσμα του demo αποθηκεύτηκε στο:

```
outputs/traced_agent_demo_q28.md
```

Η χρήση του Langfuse βοηθά στην κατανόηση της εσωτερικής ροής του συστήματος, καθώς επιτρέπει την παρακολούθηση επιμέρους βημάτων, χρόνων εκτέλεσης και LLM interactions. Στην παρούσα εργασία το Langfuse χρησιμοποιήθηκε σε demo επίπεδο και όχι ως πλήρως ενσωματωμένο tracing layer σε κάθε script του project.

Αυτό αποτελεί χρήσιμο σημείο για μελλοντική βελτίωση, καθώς η πλήρης ενσωμάτωση observability στο βασικό agent flow θα επέτρεπε πιο συστηματική ανάλυση κόστους, latency, tool calls και failure cases.

13. Production setup / Docker

Για το production setup χρησιμοποιήθηκε Docker Compose για την εκκίνηση του Qdrant vector store. Το σχετικό αρχείο είναι:

```
docker-compose.yml
```

Το Qdrant εκτελείται ως container και εκθέτει τα ports:

```
6333
6334
```

Η εκκίνηση γίνεται με:

```
docker compose up -d
```

Η λειτουργία του container ελέγχθηκε με:

```
docker compose ps
```

και επιβεβαιώθηκε ότι το Qdrant container ήταν ενεργό. Επιπλέον, το endpoint:

```
curl http://localhost:6333/collections
```

επέστρεψε τις δύο indexed collections:

```
sklearn_rag_v1_fixed
sklearn_rag_v2_structured
```

Αυτό επιβεβαιώνει ότι το vector store λειτουργεί και ότι οι δύο εκδοχές του corpus έχουν indexed embeddings διαθέσιμα για retrieval.

Στην παρούσα υλοποίηση το Docker setup καλύπτει κυρίως το Qdrant service. Η Python εφαρμογή εκτελείται τοπικά μέσω virtual environment και `requirements.txt`. Αυτό σημαίνει ότι το σύστημα είναι αναπαραγωγίσιμο σε local development περιβάλλον, αλλά δεν έχει γίνει πλήρης containerization όλης της Python εφαρμογής με ξεχωριστό Dockerfile.

Η επιλογή αυτή είναι αποδεκτή ως πρακτικό local setup, αλλά αποτελεί περιορισμό ως προς το πλήρες production packaging. Μελλοντικά, θα μπορούσε να προστεθεί Dockerfile για την Python εφαρμογή, ώστε ολόκληρο το σύστημα να εκτελείται με ενιαίο `docker compose up`.

14. Limitations και επόμενα βήματα

Παρότι το σύστημα υλοποιεί ολοκληρωμένο Agentic RAG workflow, υπάρχουν ορισμένοι περιορισμοί.

14.1 Περιορισμοί

1. **Περιορισμένο corpus** Το corpus αποτελείται από 10 επιλεγμένες σελίδες του scikit-learn documentation. Αυτό επιτρέπει εστιασμένη αξιολόγηση, αλλά σημαίνει ότι το σύστημα δεν καλύπτει όλο το scikit-learn.

2. **Rule-based baseline agent** Ο αρχικός agent βασίζεται σε deterministic keyword routing. Αυτό τον κάνει σταθερό και εύκολα αξιολογήσιμο, αλλά όχι πλήρως agentic. Για τον λόγο αυτό προστέθηκε LangGraph ReAct agent ως βελτιωμένη έκδοση.
3. **LangGraph routing dependence** Ο LangGraph agent επιλέγει εργαλείο με LLM-based routing. Αυτό είναι πιο agentic, αλλά μπορεί θεωρητικά να οδηγήσει σε λάθος tool selection. Για αυτό υποστηρίζεται και hybrid mode με fallback στον rule-based router.
4. **Offline HAIC evaluation** Το HAIC benchmarking βασίζεται σε offline proxy accept/reject και όχι σε πραγματικό user study. Επομένως, οι μετρικές HAIC πρέπει να ερμηνεύονται ως προσεγγιστική αξιολόγηση και όχι ως πραγματική ανθρώπινη αξιολόγηση.
5. **Demo-level Langfuse integration** Το Langfuse tracing υλοποιήθηκε σε ξεχωριστό demo script και όχι πλήρως στον κύριο agent flow.
6. **Partial Dockerization** Το Docker Compose εκκινεί το Qdrant vector store, αλλά η Python εφαρμογή τρέχει τοπικά μέσω virtual environment. Δεν υπάρχει ακόμα πλήρες Dockerfile για όλη την εφαρμογή.
7. **Network dependencies** Ορισμένα ingestion βήματα, όπως το structured chunking, μπορούν να εξαρτώνται από πρόσβαση στο scikit-learn.org.

14.2 Επόμενα βήματα

Μελλοντικές βελτιώσεις θα μπορούσαν να περιλαμβάνουν:

- επέκταση του corpus σε περισσότερες ενότητες του scikit-learn documentation,
- πλήρη ενσωμάτωση Langfuse tracing στον κύριο agent flow,
- Dockerfile για πλήρη containerization της Python εφαρμογής,
- ξεχωριστή αξιολόγηση του LangGraph routing σε μεγαλύτερο test set,
- προσθήκη hybrid search ή reranking,
- πραγματικό user study για πιο αξιόπιστη HAIC αξιολόγηση,
- καλύτερη διαχείριση configuration μέσω ενιαίου config αρχείου.

15. Δήλωση συνεισφοράς μελών

Η εργασία υλοποιήθηκε ως ατομικό project. Η ανάπτυξη περιλάμβανε επιλογή corpus, ingestion, chunking, indexing στο Qdrant, υλοποίηση RAG pipeline, agent routing, LangGraph ReAct agent, evaluation, HAIC benchmarking, Langfuse demo, README και τεχνική τεκμηρίωση.

Η τελική υλοποίηση περιλαμβάνει:

- δύο ingestion/chunking strategies,
- δύο Qdrant collections,
- βασικό RAG pipeline,
- rule-based agent baseline,

- LangGraph ReAct agent,
- golden test set 30 ερωτήσεων,
- retrieval και LLM-as-judge evaluation,
- Tool Call Accuracy,
- HAIC benchmarking,
- Langfuse demo,
- Docker Compose setup για Qdrant,
- πλήρες README και τεχνική αναφορά.

Συνολικά, το project υλοποιεί ένα λειτουργικό Agentic RAG σύστημα για το scikit-learn documentation, με έμφαση στην τεκμηριωμένη παραγωγή απαντήσεων, την αξιολόγηση και την αναπαραγωγικότητα.